

Deployment Profiles: Evaluated Configurations from Part 2

Deployment Profiles: Evaluated Configurations from Part 2

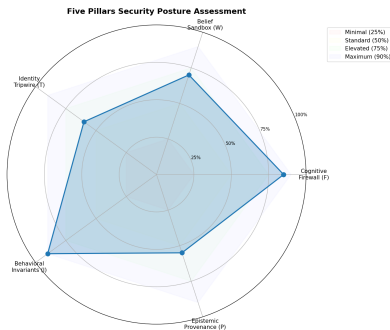


Figure 1: Five-Pillar operator posture assessment radar (Cognitive Firewall, Belief Sandbox, Identity Tripwire, Behavioral Invariants, Provenance), color-coded against readiness thresholds.

Deployment Profiles: Evaluated Configurations from Part 2

II

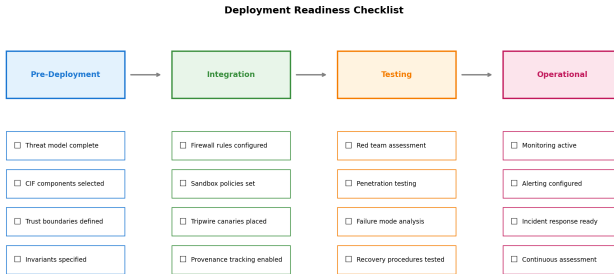


Figure 2: Pre-deployment → Integration → Testing → Operational checklist flowchart mapping CIF enforcement points to deployment phases.

Deployment Profiles: Evaluated Configurations from Part 2

III

In Part 2, we evaluated specific configurations of the Cognitive Integrity Framework to understand how different tuning parameters affected security and performance outcomes. The following profiles are derived directly from the **Parameter Sensitivity Analysis** (Part 2, Section 5.3) and **Architecture-Specific Results** (Part 2, Section 5.1).

Profile A: The “Internal Tool” Baseline (Low Latency) I

This profile corresponds to the “High Usability” configuration tested in the sensitivity analysis ($\delta = 0.95$). It is designed for low-risk, human-in-the-loop environments.

Configuration Parameters:

- ▶ **Trust Decay (δ):** 0.95. Maintained $>50\%$ trust retention even after 13 delegation hops.
- ▶ **Firewall Sensitivity:** Relaxed ($\tau = 0.9$).
- ▶ **Consensus:** Simple Majority.

Observed Performance (Part 2, Parameter Sensitivity Analysis):

- ▶ **Latency Overhead:** Minimal ($\sim 15\%$ baseline).
- ▶ **Detection Rate:** **87%** (vs 94% optimal).
- ▶ **Trade-off:** The high trust decay allows for fluid, deep delegation chains but increases vulnerability to subtle trust laundering (Ω_4).

Profile B: The “Customer Facing” Baseline (Balanced) I

This profile corresponds to the **Empirically Optimal Configuration** identified in Part 2 (Architecture-Specific Results), which balances security guarantees with operational overhead.

Configuration Parameters:

- ▶ **Trust Decay (δ):** 0.80. At this setting, trust degrades to <50% after 3 hops, strictly bounding the “radius of effective delegation.”
- ▶ **Firewall Sensitivity:** Balanced ($\tau = 0.5$).
- ▶ **Consensus:** Variable (Architecture Dependent).

Observed Performance (Part 2, Parameter Sensitivity Analysis):

- ▶ **Latency Overhead:** Reduced detection latency (~ 8.5 s for drift detection).
- ▶ **Detection Rate:** 94%.

Profile B: The “Customer Facing” Baseline (Balanced) II

- ▶ **Resilience:** Maximizes the F1 score, providing the best empirically observed trade-off between False Positives (0.06) and True Positives.

Profile C: The “Autonomous Operator” Baseline (High Assurance) I

This profile corresponds to the “Byzantine-Heavy” configuration tested in Part 2 (Byzantine Consensus Analysis). It is required for high-stakes, unsupervised environments.

Configuration Parameters:

- ▶ **Trust Decay (δ):** 0.60. Aggressive decay. Trust halves every 1.3 hops, enforcing a strictly flat command structure.
- ▶ **Firewall Sensitivity:** Strict ($\tau = 0.4$).
- ▶ **Consensus:** Byzantine Fault Tolerance ($n \geq 3f + 1$).

Observed Performance (Part 2, Parameter Sensitivity Analysis):

- ▶ **Latency Overhead:** Significant (+35-48% depending on corroboration count).
- ▶ **Detection Rate:** **95%** (general) to **98%** (LangGraph-specific).

Profile C: The “Autonomous Operator” Baseline (High Assurance) II

- ▶ **Resilience:** Specifically tunes parameters to resist coordination attacks (Ω_4), accepting higher latency for a near-zero semantic drift rate.

Architecture-Specific Observations I

Beyond the parameter profiles, Part 2's architecture adapters revealed specific interactions between the defense framework and the underlying agent topology.

LangGraph (State Machines)

Observation: LangGraph architectures achieved the highest overall detection rates (98%) in our tests (Part 2, Architecture-Specific Results). **Mechanism:** The explicit definition of state transitions allowed for rigorous **Invariant Checking**. Invalid state transitions were detected deterministically by the framework.

Architecture-Specific Observations II

CrewAI (Role-Based)

Observation: CrewAI architectures performed best against “Trust Exploitation” attacks (94% detection) (Part 2, Architecture-Specific Results). **Mechanism:** The framework’s role definitions acted as implicit **Identity Tripwires**. When an agent attempted to act outside its defined role, the behavior was flagged as a role violation.

Minimal Viable Implementation I

We also evaluated a “Minimal Viable Implementation” (MVI) to determine the baseline efficacy of the framework’s core components.

The Setup:

1. **Trust Decay:** $\delta = 0.80$ (The optimal balance point).
2. **Cognitive Firewall:** Ingress only.
3. **Tripwires:** One per agent.

Result: Even this minimal setup shifted the success rate against low-effort attacks from 100% (Baseline) to <5%, providing a critical first line of defense.

Alignment with Emerging Standards I

Practitioners deploying cognitive security must increasingly demonstrate compliance with industry and government standards. CIF's defense mechanisms map directly to two major 2025–2026 standardization efforts.

OWASP Top 10 for Agentic Applications (2026)

The OWASP Agentic Top 10 identifies ten risks (ASI01–ASI10) specific to autonomous multiagent deployments. CIF addresses the majority through its layered defense architecture:

OWASP Risk	CIF Defense	Profile Coverage
ASI01: Agent Goal Hijack	Cognitive Firewall + Tripwires	All profiles
ASI02: Tool Misuse/Exploitation	Belief Sandbox (tool output isolation)	Profiles B, C
ASI03: Identity/Privilege Abuse	Trust Calculus (δ^d decay)	All profiles
ASI06: Memory/Context Poisoning	Tripwire monitoring + Drift detection	Profiles B, C
ASI07: Insecure Inter-Agent Comm.	Provenance attestation	Profiles B, C
ASI08: Cascading Failures	Byzantine Consensus	Profile C
ASI10: Rogue Agents	Full CIF stack	Profile C

Alignment with Emerging Standards II

NIST Zero Trust Architecture for AI Agents

NIST's extension of SP 800-207 to AI agents establishes “never trust, always verify” principles. CIF operationalizes zero trust for cognitive interactions:

- ▶ **Continuous verification:** Every inter-agent message is evaluated by the Cognitive Firewall
- ▶ **Micro-segmentation:** Beliefs from external sources are sandboxed before integration
- ▶ **Least privilege:** Trust scores decay exponentially with delegation depth (δ^d)
- ▶ **Continuous authentication:** Provenance attestation provides cryptographic message origin tracking

Profile A (Internal Tool) provides partial NIST alignment. Profile B (Customer Facing) achieves substantial compliance. Profile C (Autonomous Operator) is the profile that maps most completely to the controls cited in the OWASP and NIST frameworks above; treat any mapping as design intent, not certification.

CIF Composer: Interactive Deployment Planning Tool I

Part 2 ships an **interactive CIF Composer web UI** (`output/web/cif_composer.html`) that can assist deployment planning before committing to a production configuration. The Composer is a self-contained HTML/JS/D3 application requiring no server — open it directly in a browser from the Part 2 repository.

Key capabilities:

Feature	Description
8-module palette	Drag-and-drop Cognitive Firewall, Belief Sandbox, Tripwires, Drift Detection, Trust Calculus, Provenance, Byzantine Consensus, Invariant Checker
Canvas composition	Wire modules in series, parallel, or hybrid configurations visually
Live metric computation	Detects and computes composite detection rate in real time using Theorems 3.1/3.2 from Part 1
Category law verification	Verifies the Defense Category laws (identity, associativity) for the current pipeline composition
4 deployment presets	Loads Profiles A, B, C, and the Minimal Viable Implementation (MVI) directly
Export	Generates Python SDK configuration code, JSON pipeline spec, and SVG diagram of the composed architecture
Category Explorer tab	9 interactive D3 diagrams for commutative diagrams, Hasse lattices, operadic trees, Kleisli flows, and lens diagrams

CIF Composer: Interactive Deployment Planning Tool II

Workflow for operators: (1) Open `output/web/cif_composer.html` from the Part 2 repository. (2) Load the profile preset closest to your deployment context (Profile A/B/C). (3) Customize by adding, removing, or reordering modules. (4) Observe the live detection rate estimate and verify category laws. (5) Export the Python configuration and paste into your deployment scaffold. This replaces manual parameter lookup in tables with an interactive, law-verified design session.

Note: *The Composer's detection rate estimates are derived from the parametric simulation in Part 2. They reflect fully-mature (Level-5) adapter performance. For current Level-3 adapter baselines, apply the adapter-maturity discount discussed in §3.*